

# Concorrência e transações

Simulação com duas sessões e garantias usadas nas operações reais

## O estado que precisa ser preservado

Duas requisições podem consultar a mesma combinação livre antes de qualquer uma confirmar. Um SELECT prévio não impede a corrida, pois cada transação enxerga apenas dados confirmados. A regra final está no banco: **UNIQUE (id\_residente, dia\_semana, turno)**. Assim, o mesmo residente não ocupa duas unidades no mesmo plantão.

**concorrencia.py** abre duas threads, cada uma com sessão e transação próprias. Uma barreira aproxima o instante da disputa e SET LOCAL lock\_timeout = '4s' impede que a requisição espere sem limite. O endpoint **POST /concorrencia/simular** devolve os logs na ordem em que foram registrados; os tempos variam em cada execução.

## Cenário 1: sem lock explícito

As duas sessões consultam a escala e podem receber "livre". Ambas tentam inserir. Uma confirma; a outra espera a resolução da chave única e recebe a violação 23505. A inconsistência não entra no banco, mas o erro aparece como conflito de constraint. O cenário demonstra por que a consulta antecipada não substitui a restrição.

## Cenário 2: lock pessimista

Cada sessão pede **SELECT ... FOR UPDATE** na linha do residente antes de consultar a escala. Uma obtém o lock e a outra aguarda. Depois do primeiro COMMIT, a segunda lê o estado atualizado e desiste com mensagem de negócio, sem tentar o INSERT.

```
select(Residente)
    .where(Residente.id_profissional == ID_RESIDENTE)
    .with_for_update()
```

## Cenário 3: versão otimista

Uma escala de teste é criada e as duas sessões carregam a mesma versão. Cada uma tenta mudar id\_unidade para um valor diferente. O SQLAlchemy inclui id\_escala e versao no WHERE do UPDATE. A primeira grava e incrementa a versão; a segunda afeta zero linhas e recebe StaleDataError. A alteração confirmada não é sobrescrita.

| Estratégia | Momento do conflito      | Efeito para a segunda sessão            |
|------------|--------------------------|-----------------------------------------|
| Restrição  | Na gravação              | Recebe violação de unicidade.           |
| Pessimista | Antes da leitura crítica | Espera e depois vê o novo estado.       |
| Otimista   | No UPDATE                | Falha pela versão e precisa recarregar. |

## Limpeza por identidade

Cada cenário guarda o `id_escala` retornado pelo flush. No finally, `_limpar_ids` remove somente esses identificadores. Residente, dia e turno nunca são usados como filtro de limpeza, pois podem coincidir com uma escala legítima criada enquanto a demonstração está rodando.

**Regra operacional.** A simulação restaura apenas os registros que ela própria criou. Um teste deixa uma escala legítima com a mesma combinação de negócio e confirma que essa linha sobrevive.

## Proteções usadas fora da demonstração

| Operação                 | Mecanismo                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------|
| POST /escalas/reajustar  | <code>sp_reajustar_escala</code> bloqueia Residente antes de ler a origem e incrementa versão.       |
| PUT /escalas/{id_escala} | O cliente envia a versão lida; divergência ou <code>StaleDataError</code> retorna HTTP 409.          |
| Média de procedimento    | Deltas atômicos e <code>pg_advisory_xact_lock</code> serializam a manutenção dos acumuladores.       |
| Papel profissional       | A chave primária de <code>Papel_Profissional</code> serializa a reserva entre preceptor e residente. |
| Horário do procedimento  | FOR UPDATE no atendimento pai fecha a corrida entre as duas tabelas.                                 |

## Testes adversariais

`tests/test_adversarial.py` usa conexões independentes e cobre cinco falhas de concorrência: limpeza seletiva, inserção simultânea em unidades diferentes, duas chamadas de reajuste, atualização concorrente da média e reserva simultânea de papéis. A suíte também executa os três cenários da simulação e confere que todos preservam a regra.

Os testes rodam no PostgreSQL descartável publicado somente em **localhost:5434**. A configuração recusa porta 5432, host remoto e outro SGBD antes de importar a aplicação. Pela linha de comando, **python demo\_concorrencia.py** imprime os três desfechos e retorna código zero quando todos evitam a inconsistência.